

EST-1141

Lenguajes de Programación

Juan Zamora O.

Introducción a los Lenguajes de Programación



PONTIFICIA
UNIVERSIDAD
CATÓLICA DE
VALPARAÍSO

Estructura

- 1 Agenda
- 2 Paradigmas Computacionales
- 3 Criterios de evaluación de lenguaje
- 4 Implementación de lenguajes
- 5 Resumen

Agenda

- ¿Porque estudiar lenguajes de programación (LP)?
- ¿Qué es un LP?
- Paradigmas computacionales

¿Porque estudiar lenguajes de programación (LP)?

- Mayor capacidad de expresar ideas
- Mayor sustento para escoger un lenguaje
- Mejora en la habilidad de aprender nuevos lenguajes
- Mayor comprensión respecto de la implementación de los lenguajes

Brecha semántica humano-computador

- Queremos modelar el mundo real
- A menudo, lo que queremos hacer es mucho más claro que el cómo hacerlo
- La máquina solo puede manipular secuencias de 0's y 1's
 - Solo entiende instrucciones (el Cómo) de bajo nivel...lejanas a lo que queremos hacer

Un lenguaje de programación ...

- disminuye esta brecha
- entregando notación de alto nivel (más cercana a lo concebido por nosotros) que puede ser ejecutada por el computador

¿Que es un LP?

Definición Sistema notacional que permite describir el cómputo de manera legible tanto para la máquina como para el ser humano

- Cómputo entendido como un lenguaje aceptado por una máquina de Turing

¿Qué lenguaje es legible para un computador?

- Gramática libre de contexto
 - Reglas recursivas usadas para generar patrones de cadenas

¿Qué lenguaje es legible para un ser humano?

- Es la propiedad más importante hoy en día
- Un programa debe ser “facilmente” entendible por una persona distinta a quien lo construyó
- Depende mucho de la selección de abstracciones
 - De datos (int, str, class ...)
 - De control (if, while, ...)

Paradigmas Computacionales

- LP's comenzaron imitando las operaciones del computador
- Existe un efecto del funcionamiento del computador sobre el diseño de un LP
 - Variables para representar ubicaciones de memoria
 - Asignaciones para cambiar valores
 - Ejecución secuencial de instrucciones
- El modelo de cómputo predominante es el de Von Neumann

El modelo de Von Neumann (1940)

- Cableado permanente con un conjunto pequeño de operaciones de propósito general
- Operador ingresa series de códigos binarios que reemplazan el re-cableado manual de modelos anteriores
- Estas instrucciones en ***lenguaje de máquina*** son almacenadas en una memoria
- En síntesis: Unidad de procesamiento ejecuta instrucciones que operan sobre valores almacenados en memoria

- Ejecución secuencial de instrucciones
- Uso de variables representando ubicaciones de la memoria
- Uso de la asignación para modificar el contenido almacenado en esas ubicaciones

Estas 3 operaciones caracterizan a un **lenguaje imperativo**

- ¿Es posible aprovechar el paralelismo con ese modelo?
- Existen otras alternativas de declarar el cómputo de una manera menos dependiente del modelo de Von Neumann

Paradigmas Computacionales

Hay varias clases de LP denominadas paradigmas

- **Imperativo:** Instrucciones secuenciales. Variables, asignaciones y ciclos.
- **Orientado a objetos:** Centrado en los datos. Objetos representan a datos (instancias) y se comunican usando mensajes.
- **Funcional:** No incluye variables, No tiene un control secuencial
- **Lógico:** Afirmaciones son los datos más básicos.

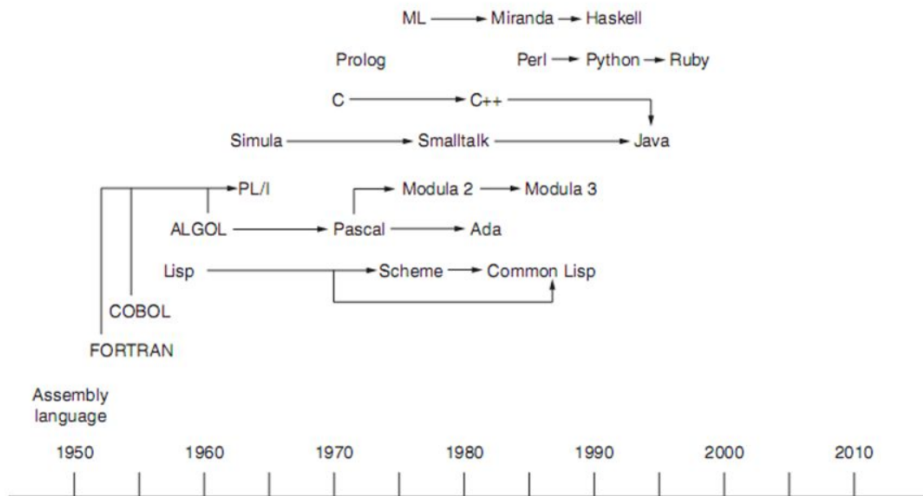
Lenguajes y Paradigmas

- Imperativo: C, Fortran ... casi todos lo soportan en algún grado
- OO: Python, R, C++, JAVA
- Funcional: Lisp, Haskell
- Lógico: Prolog

Paradigmas rara vez son puros

- El LP C hace uso de estilo funcional. E.g. gcd.
- JAVA utiliza partes imperativas y tiene tipos de datos que no son Objetos.
- Algunas excepciones: Haskell (funcional) y Smalltalk (OOP)

Línea de tiempo histórica



Criterios de evaluación de lenguaje

Lectorabilidad: La facilidad con la que los programas son leídos y entendidos. Especialmente importante para la mantención y corrección de programas. Aspectos que componen esta propiedad:

- Simplicidad: Cantidad manejable de características y constructos. Multiplicidad de características mínima. Uso adecuado de sobrecarga de operadores.
- Ortogonalidad: Que tanto se pueden combinar los constructos básicos (cantidad de reglas especiales o excepciones). Por ejemplo, una función puede retornar cualquier tipo, excepto un arreglo.

- Tipos de datos: Tipos predefinidos adecuados y facilidad para definir tipos adecuados. Ej. La no existencia del tipo **bool** y con ello el uso de un valor **int** con 0 y 1.
- Consideraciones de sintáxis: Existencia de palabras reservadas, cómo se indica el inicio y fin de una sentencia compuesta (e.g. **for**) o que tanto significado tienen estas palabras (e.g. **if**).

Escriturabilidad: La facilidad con el lenguaje puede usarse para desarrollar programas. Aspectos que componen esta propiedad:

- Simplicidad: Número reducido de constructos primitivos y un conjunto consistente de reglas para combinarlos.
- Soporte de abstracción: Capacidad de definir y usar estructuras complejas u operaciones de manera tal que se permita ignorar detalles de implementación.
- Expresividad: Conjunto relativamente conveniente de formas para especificar operaciones. Esto hace posible el desarrollo de operaciones complejas en pocas líneas de código.

Confiabilidad: Capacidad de un programa de cumplir con sus especificaciones bajo todas las condiciones. Aspectos que componen esta propiedad:

- Verificación de tipos: Capacidad de probar errores de tipo ya sea en tiempo de compilación o ejecución. Es deseable lo primero porque no hay que esperar a ejecutar el programa para identificar errores. Ej. los tipos de los parámetros pasados a las funciones.
- Aliases: Capacidad de poder definir dos nombres distintos referenciando ambos a la misma posición de la memoria. Es algo bien común en lenguajes como Python debido a eficiencia, pero daña la confiabilidad.

Costo: Costo general de usar un lenguaje.

- Entrenamiento de programadores (es una función de simplicidad)
- Escritura y mantención (función de Lecturabilidad y escritur.)
- Ejecución de programas: Verificaciones en tiempo de compilación generan programas que corren más rápido.
- Falta de confiabilidad: Fallas en software generan daños costosos.

Balances en el diseño de lenguajes

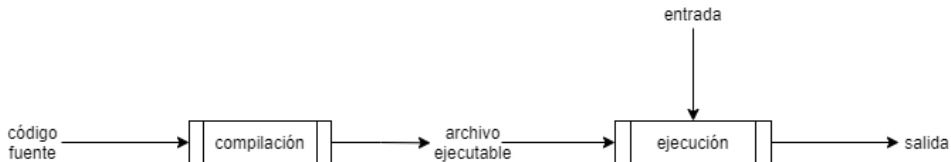
- Confiabilidad vs Costo de la ejecución
- Lecturabilidad vs Escriturabilidad
 - Lenguaje donde se pueden expresar operaciones complejas de manera concisa tiende a ser difícil de leer
- Flexibilidad para romper la abstracción vs Confiabilidad -El acceso a operaciones de bajo nivel (e.g. punteros) es más propenso a generar programas con fallos/*bugs*

Implementación de lenguajes

- Consideremos que los computadores solamente pueden ejecutar código de máquina
- La ejecución de código en cualquier otro lenguaje requiere una **traducción** al código de máquina
- Existen 3 métodos de traducción: Compilación, Interpretación y el Híbrido de ambos.

Compilación

- traduce cada unidad de compilación (archivo, módulo, clase según el lenguaje) en un módulo de objeto
- El módulo de objeto contiene código objeto, es decir código de máquina pero incompleto.
- Este módulo es completado por el enlazador (*linker*), el que resuelve las referencias

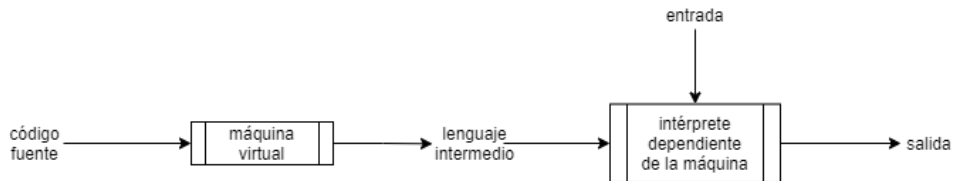


Interpretación

- El interprete es un programa que simula un computador cuyo “lenguaje de máquina” es el lenguaje interpretado (e.g. R, Python).
- Un computador puede ser visto como un interprete de su propio código de máquina implementado en HW

Hibrido

- Cada programa construido en el lenguaje es llevado a un lenguaje intermedio distinto al código de máquina.
- Luego el código intermedio es interpretado (e.g. Java bytecode y la Java Virtual Machine)
- Es más rápido que la interpretación



Compilación **Just In Time**

- Compilación al momento de ejecutar un programa
- En lugar de interpretar directamente el código en un lenguaje intermedio, lo compila en código de máquina
- Usa considerablemente más memoria y CPU inicialmente, pero resulta en ejecuciones mucho más rápidas

Resumen

- El estudio de lenguajes de programación es importante por:
 - Aumenta nuestra capacidad de resolver problemas al contar con mayor diversidad de herramientas para especificar el cómputo
 - Nos permite escoger lenguajes de manera más inteligente
 - Nos ayuda a aprender lenguajes más fácilmente
- Los principales criterios de evaluación de LP son:
 - Lecturabilidad, Escriturabilidad, Confiabilidad y Costo
- Los principales métodos para implementar LP son:
 - Compilación, Interpretación e Híbridos